

Claude code

Context limit is calculated by token.

Context is all the information available to understand something correctly in a particular session.

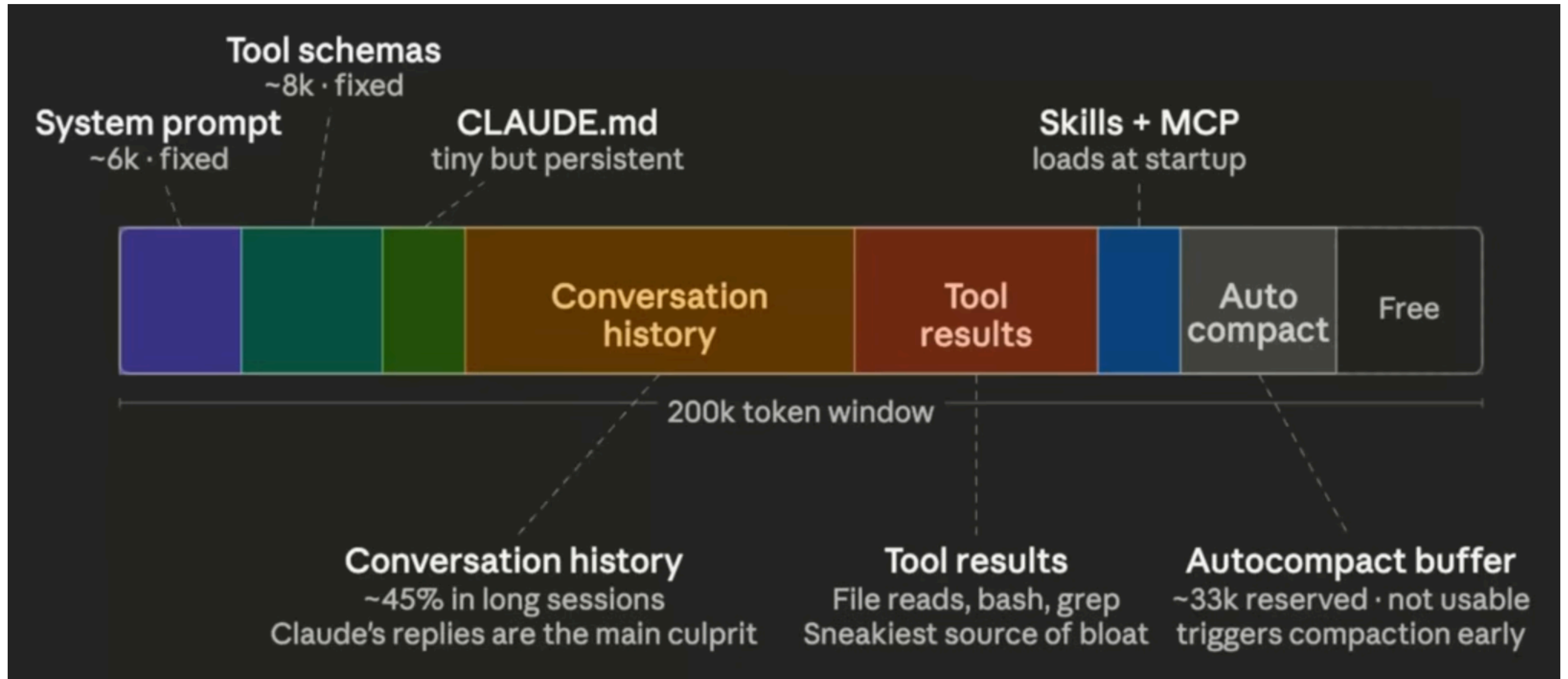
Tokens get consumes from inputs and response both known as input tokens and output tokens.

Every request sends entire conversation history from scratch. Thus, we need to be mindful about our prompts

Single long session V/s Fresh session per feature

Note: Subagents get their own isolated context window completely separate from the main session. this is how subagents helps us to save in token cost

Also instead of returning the entire history subagents only returns a summary version of it to the main context.



Representing how tokens are reserved for every specific task

After reaching context window output response may start degrading

/context : (tells about our tokens and their usage)

```
> /context
```

L Context Usage

gemma4:31b-cloud

38.9k/200k tokens (19%)

Estimated usage by category

System prompt: 2k tokens (1.0%)

System tools: 35.1k tokens (17.5%)

Skills: 1.8k tokens (0.9%)

Free space: 128.1k (64.1%)

☒ Autocompact buffer: 33k tokens (16.5%)

Auto-compact window: 200k tokens

When context window reaches to near completion auto compact action gets trigger that summarizes the earlier history to free some space in context window.

Repeated auto compaction causes corruption.

If context gets filled and cannot even store summary it hard stops the session.

/compact : (Manually compacting the context)

Auto compaction that claude will do will happen automatically but not as per our own time it may happen any time. Thus, use this command proactively.

Use **.claudeignore** to keep irrelevant files out of context from claude.

CLAUDE.md

- Claude code cant remember things from prev sessions
- Need to repeat instructions across sessions
- Thus leads to inconsistent code generation

Soln :-> Claude .md

claude.md acts a persistent system prompt for our project

claude.md file is a special project-level instruction file used by claude code to guide how it behaves while working on your codebase

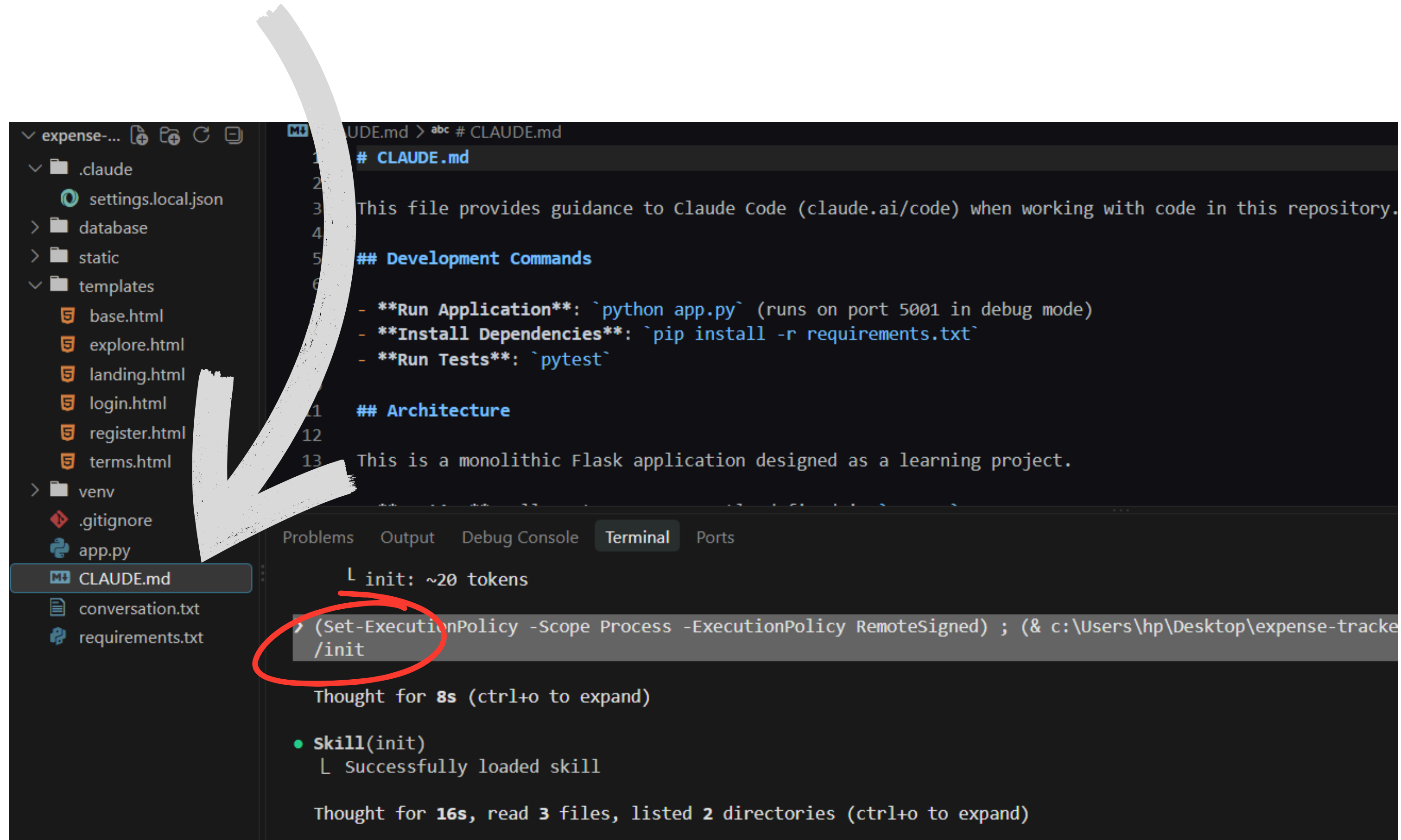
/init : (helps to create a claude.md file by scanning the entire codebase)

We can also create claude.md manually but /init is preferred.

Situation	Why <code>/init</code> helps
Onboarding an existing codebase you didn't write	Faster than reading everything yourself
Large repos with many files	Claude can spot patterns you might forget to document
You're new to CLAUDE.md	Good way to see what the format looks like
Quick prototypes	Don't want to invest time writing docs

CLAUDE.md should not be very long to be precise it should not exceed 200-300 lines. then output starts degrading.

Creating claude.md from /init



/init

- /init is a good starting point but in complex setup generated CLAUDE.md is roughly 30% usefull.
- Remaining 70% workflows,constraints,what to avoid,deployment targets,naming conventions we have to write.
- CLAUDE.md must include “Project Context”
- Explain code/project architecture
- Describe coding conventions and style to be used.
- Constraint claude about what tools/frameworks to use.
- Add commands to run for testing,running and deploment.
- Highlight possible edge cases,critical warnings,not to do list.(very important)

.claude Folder

Acts as Claude Codes's local configuration directory that controls how claude behaves-either for a specific project or across all projects on your machine

Project-level `.claude/` — scoped to one project, committed to the repo, shared with the team.

Global/User-level `~/.claude/` — scoped to your machine, applies to every project, personal to you.

.claude folder contains **a) commands/** folder where we write our custom commands **b) settings.json** **c) skills/** **d) agents/** (here we define our subagents if we are creating any.

Claude.md

Project Root ---- ./CLAUDE.md

Inside .claude folder --- ./claude/CLAUDE.md

./CLAUDE.local.md Local(Personal,Gitignored)

Claude reads ./CLAUDE.local.md alongside the main CLAUDE.md and its automatically gitignored so your personal tweaks never land in the repo
(It considers our personal preferences and workflows at project level)

GLOBAL ---- ~/.claude/CLAUDE.md

Your personal preferences that apply across all projects

Subdirectory ---- ./some/folder/CLAUDE.md

Convenient for large repositories so for that particular folder claude code will read this particular claude.md to perform task

Split into .claude/rules/ files

```
# CLAUDE.md
## Code style
...20 lines...
## Testing
...20 lines...
## Security
...20 lines...
## API conventions
...20 lines...
```

```
.claude/rules/
├── code-style.md
├── testing.md
├── security.md
└── api-conventions.md
```

Here, claude is intelligent enough, it does not automatically load every rule for a fresh session; instead, it will load as per requirement (lazy loading).

Use @ imports to reference external docs

```
## API Conventions
```

```
See @docs/api-guidelines.md
```

```
## Git Workflow
```

```
See @docs/contributing.md
```

Use subdirectory CLAUDE.md files

Claude Auto Memory Feature

Auto memory is a persistent directory where Claude records learnings, patterns, and insights as it works.

When Claude discovers something about your project (“oh, this application uses INR instead of USD”), it saves that to auto-memory. Next session, it already knows. No more repeating yourself.

It lives in `~/.claude/projects/<project>/memory/`:

Claude store this auto memory feature in memory.md

/memory (command)

> /memory

Memory

Auto-memory: on

- 1. Project instructions Saved in ./CLAUDE.md
- 2. User instructions Saved in ~/.claude/CLAUDE.md
- > 3. Open auto-memory folder ✓

Learn more: <https://code.claude.com/docs/en/memory>

Enter to confirm · Esc to cancel